Отчёт по индивидуальному заданию №2

Многопоточное моделирование работы супермаркета

Исполнитель: Батышкина Алёна Игоревна

Группа: БПИ253

Вариант: 8

Дата: 24.10.2026

---

1. Условие задачи

Разработать многопоточную имитационную программу на языке C, моделирующую процесс обслуживания покупателей в супермаркете. Программа должна использовать библиотеку `pthread` и синхропримитивы (мьютексы и семафоры) для согласованного асинхронного взаимодействия потоков, обеспечивать наглядное представление результатов в реальном времени и корректно завершаться при любых входных данных, включая прерывание по сигналу.

Ключевые сущности предметной области:

- Покупатели, прибывающие в супермаркет в случайные моменты времени;
- Кассы с очередями ограниченной вместимости;
- Кассиры, обслуживающие покупателей.

Параметры модели задаются через конфигурационный файл и включают количество покупателей, вместимость очередей, временные интервалы, стратегию выбора очереди, продолжительность рабочего дня и множитель времени (`time_scale`) для регулировки скорости симуляции.

---

2. Сценарий моделирования

2.1. Отображение сущностей в потоки

В программе принята многопоточная архитектура. Каждая сущность предметной области, обладающая поведением, реализована как отдельный независимый поток POSIX (`pthread_t`).

Сущность предметной области	Представление в программе	Поведение (поток)

Сущность предметной области	Представление в программе	Поведение (поток)
Покупатель	структура `Customer` + `pthread_t`	Отдельный поток: имитация выбора товаров (`usleep`), захват мьютексов очередей для выбора стратегии, ожидание обслуживания (`sem_wait` / `pthread_cond_wait`), завершение.
Кассир	структура `Cashier` + `pthread_t`	Отдельный поток: бесконечный цикл ожидания клиента, обслуживание (`usleep`), сигнализация клиенту об окончании (`sem_post` / `pthread_cond_signal`).
Генератор	функция `generator_thread_func`	Отдельный поток: создает потоки покупателей с заданным случайным интервалом, устанавливает флаг `entrance_closed` по исчерпанию лимита.
Дисплей	функция `display_thread_func`	Отдельный фоновый поток: периодически (каждые 50 мс) считывает состояние симуляции под защитой мьютексов и обновляет консоль через ANSI escape-последовательности.

2.2. Жизненный цикл покупателя (потока)

1. **Прибытие:** Поток-генератор инициализирует параметры покупателя (ID, количество товаров) и запускает его поток.
2. **Выбор товаров:** Поток покупателя вызывает `usleep` на случайное время из диапазона `[shopping_time_min, shopping_time_max]` .
3. **Выбор очереди:** Поток последовательно блокирует мьютексы обеих очередей (во избежание deadlock), сравнивает их размеры согласно стратегии (SHORTEST/FIRST/RANDOM) и выбирает целевую очередь.
4. **Вставание в очередь:** Поток блокирует мьютекс выбранной очереди. Если очередь переполнена (size >= capacity), поток проверяет вторую очередь и переключается в неё (с записью в лог «switches to queue»). Если обе очереди заполнены — увеличивает total_left_no_service и завершается. В противном случае добавляет свой ID в массив очереди.
5. **Пробуждение кассира:** Поток вызывает `sem_post` (или `pthread_cond_signal`) для семафора/условной переменной кассира, сигнализируя о появлении клиента.
6. **Ожидание обслуживания:** Поток блокируется на `sem_wait` (или `pthread_cond_wait`), ожидая сигнала от кассира.
7. **Завершение:** Кассир обслуживает покупателя и вызывает сигнал. Поток покупателя разблокируется, обновляет свое время ожидания и корректно завершается.

2.3. Взаимодействие потоков

В отличие от последовательной версии (ИДЗ-1), где события хранились в min-куче и обрабатывались в одном потоке, в многопоточной версии (ИДЗ-2) все сущности работают параллельно. Синхронизация осуществляется через мьютексы (для защиты общих данных от гонок) и семафоры/условные переменные (для сигнализации о событиях между потоками).

2.4. Механизмы синхронизации

Для обеспечения корректного асинхронного взаимодействия потоков использованы следующие примитивы:

1. **Мьютексы (`pthread_mutex_t`)**:
   - `sim->stats.mutex` : защищает глобальную статистику от гонок данных (data race) при одновременном обновлении разными потоками.
   - `sim->queues[i].mutex` : защищает массив `customer_ids` и счетчик `size` очереди при добавлении/удалении покупателей.
   - `rng_mutex` (в `rng.c`): защищает вызов `rand()` от одновременного доступа нескольких потоков покупателей, обеспечивая потокобезопасность генератора случайных чисел.
2. **Семафоры (`sem_t`) / Условные переменные (`pthread_cond_t`)**:
   - `sim->queues[i].cashier_signal` / `item_available` : кассир ждёт появления покупателя в своей очереди. Сигнализируется покупателем после успешного вставания в очередь.
   - `cashier->service_done` / `service_cond` : покупатель ждёт завершения обслуживания.
   - `sim->cashiers[i].mutex` : защищает поля is_sleeping и current_customer_id (для корректного отображения на дисплее).
   - `sim->global_mutex` : защищает current_time, total_arrived и флаг entrance_closed.

---

3. Архитектура программы

3.1. Структура проекта

```
project/
├── include/                    Заголовочные файлы
│   ├── types.h                 Базовые типы, структуры, pthread, sem_t
│   ├── config.h                Интерфейс конфигурации
│   ├── io.h                    Интерфейс ввода-вывода
│   ├── rng.h                   Интерфейс генератора случайных чисел
│   ├── simulation.h            Интерфейс управления симуляцией
│   └── display.h               Интерфейс визуализации
├── src/                        Исходные файлы
│   ├── main.c                  Точка входа, обработка SIGINT
│   ├── config.c                Загрузка и валидация конфигурации
│   ├── io.c                    Ввод-вывод через системные вызовы
│   ├── rng.c                   Потокобезопасный генератор случайных чисел
│   ├── simulation.c            Инициализация примитивов, управление жизненным циклом потоков
│   ├── threads.c               Логика потоков (кассир, покупатель, генератор, дисплей)
│   ├── display.c               Табличный интерфейс с блокировками мьютексов
│   └── types.c                 Массив имён событий
├── tests/                      Тестовые конфигурации и скрипты
│   ├── run_tests.sh            Скрипт автоматического тестирования
│   └── *.txt                   18 тестовых конфигураций
├── Makefile                    Скрипт сборки (с -lpthread и условной компиляцией)
└── report.md                   Данный отчёт
```

3.2. Модульная структура

| Модуль | Ответственность | Используемые системные вызовы и API |
|=======|================-|----------------------------------| | `io.c` | Ввод-вывод в консоль и лог-файл | `open`, `write`, `close`, `vsnprintf` | | `config.c` | Парсинг конфигурационного файла | `open`, `read`, `close` | | `rng.c` | Потокобезопасная генерация случайных чисел | `srand`, `rand`, `time`, `pthread_mutex_lock/unlock` | | `simulation.c` | Инициализация примитивов и управление жизненным циклом потоков | `pthread_create`, `pthread_join`, `pthread_mutex_init`, `sem_init` / `pthread_cond_init`, `calloc`, `free` | | `threads.c` | Логика рабочих потоков (кассир, покупатель, генератор, дисплей) | `sem_wait/post`, `pthread_cond_wait/signal`, `usleep` | | `display.c` | Табличный интерфейс с блокировками мьютексов | `printf`, ANSI escape-коды, `pthread_mutex_lock/unlock` | | `main.c` | Точка входа, обработка сигналов | `sigaction`, `write` (async-signal-safe) | | `types.c` | Массив строковых представлений событий | — |

---

4. Описание пользовательского интерфейса

4.1. Конфигурационный файл

Параметры задаются в текстовом файле в формате `ключ = значение`. Строки, начинающиеся с `#`, считаются комментариями. Пример:

```
total_customers = 50
queue_capacity = 5
shopping_time_min = 10
shopping_time_max = 30
arrival_interval_min = 2
arrival_interval_max = 5
items_min = 1
items_max = 8
item_service_time = 15
queue_strategy = 0
work_day_duration = 1800
random_seed = 42
display_delay_ms = 100
time_scale = 1
```

Параметр queue_strategy принимает значения: 0 — SHORTEST (кратчайшая очередь), 1 — FIRST (первая очередь), 2 — RANDOM (случайный выбор). Параметр time_scale — множитель реальной длительности usleep в потоках (для визуального наблюдения). Модельное время (current_time) при этом не меняется — оно идёт по сумме интервалов прибытия. Например, time_scale = 10 замедляет программу по настенным часам в 10 раз, но модельные секунды (Time: XXXX / 1800) идут с той же скоростью. Параметр display_delay_ms — задержка обновления дисплея в миллисекундах (используется в последовательной версии; в многопоточной версии задержка жёстко задана 50 мс в потоке дисплея).

4.2. Табличный интерфейс в реальном времени

Во время выполнения симуляции в терминале отображается таблица, обновляющаяся на месте без прокрутки. Отрисовка осуществляется в отдельном фоновом потоке (display_thread_func), который каждые 50 мс считывает состояние симуляции, захватывая мьютексы очередей и статистики, и выводит обновлённую таблицу с помощью ANSI escape-последовательностей. Это позволяет наблюдателю понимать происходящее в каждый момент времени без блокировки рабочих потоков.

```
|=========================================================|
| SUPERMARKET SIMULATION                                  |
| Time: 0245 / 1800                                       |
|=========================================================|
| Arrived: 28   Served: 12   Left: 16                     |
|=========================================================|
| Cashier 0: [C2][C5][C8]              serving C2         |
| Cashier 1: [C3][C7]                sleeping             |
|=========================================================|
| Recent events:                                          |
|                                                         |
|                                                         |
|                                                         |
|=========================================================|
```

Таблица содержит:

- Заголовок с текущим модельным временем и прогрессом рабочего дня;
- Сводную статистику (прибыло, обслужено, ушло);
- Состояние каждой кассы (список покупателей в очереди и статус кассира);
- Раздел «Recent events» — унаследован от последовательной версии (ИДЗ-1). В многопоточной модели события отражаются только в логе `simulation.log`, а дисплей показывает актуальное состояние очередей и кассиров в реальном времени.

4.3. Лог-файл

Параллельно с табличным интерфейсом ведётся подробный лог в файле simulation.log. Запись в лог осуществляется конкурентно из потоков покупателей, кассиров и генератора. Для предотвращения перемешивания строк (data race) используется системный вызов write() с атомарными буферами фиксированного размера (1024 байта), что гарантирует целостность каждой записи. В лог записываются все события с временными метками, детальным описанием действий потоков и итоговая статистика.

---

5. Варианты запуска и завершения программы

5.1. Способы запуска

| Способ | Команда | Поведение |

Способ	Команда	Поведение
С конфигурационным файлом	`./supermarket config.txt`	Загрузка параметров из файла
Без параметров	`./supermarket`	Использование значений по умолчанию
С невалидным файлом	`./supermarket invalid.txt`	Сообщение об ошибке, завершение с кодом 1
Альтернативная версия (условные переменные)	`./supermarket_condvar config.txt`	Запуск версии с `pthread_cond_t` вместо семафоров
Через Makefile (тестовый конфиг)	`make run-test`	Сборка и запуск с `tests/test_small.txt`
Запуск всех тестов	`make test`	Сборка и прогон 18 автоматизированных тестов

5.2. Способы завершения

Сценарий	Поведение
Нормальное завершение	Все потоки завершены, выводится статистика, проверяется закон сохранения, закрывается лог
Прерывание пользователем (Ctrl+C)	Обработчик SIGINT (через sigaction) устанавливает атомарный флаг g_interrupted и (для sem-версии) вызывает sem_post для пробуждения кассиров. Для condvar-версии пробуждение происходит по pthread_cond_timedwait с таймаутом 100 мс. Потоки самостоятельно проверяют g_interrupted в своих циклах и корректно завершаются, дообслуживая покупателей, уже стоящих в очередях.
Ошибка конфигурации	Сообщение об ошибке, завершение с кодом возврата 1
Нарушение закона сохранения	Сообщение об ошибке, завершение с кодом возврата 1

---

6. Результаты работы программы

6.1. Пример запуска

При конфигурации: 50 покупателей, вместимость очереди 5, стратегия FIRST, seed = 42.

Итоговая статистика:

```
****************************
SIMULATION STATISTICS
****************************
Simulation duration:   190 sec
Total arrived:         50
Total served:          18
Total left (no serv.): 32
Average shopping time: 14.36 sec
Max queue 0 size:      5
Max queue 1 size:      5
Cashier 0 served:      8
Cashier 1 served:      10
****************************
```

```
Conservation law verified: 50 = 18 + 32
```

6.2. Сравнение стратегий выбора очереди

Все три стратегии прогнаны на одинаковой конфигурации (`config.txt` : 50 покупателей, вместимость очереди 5, seed = 42), чтобы сравнение было корректным.

Стратегия	Обслужено	Ушло без обслуживания	Эффективность	Cashier 0 / Cashier 1
SHORTEST (`queue_strategy = 0`)	19	31	38%	9 / 10
FIRST (`queue_strategy = 1`)	19	31	38%	9 / 10
RANDOM (`queue_strategy = 2`)	17	33	34%	8 / 9

Наблюдения:

1. **Стратегии SHORTEST и FIRST показали одинаковый результат** (19 обслужено, 31 ушёл). Это объясняется реализованным механизмом переключения очередей (см. п. 2.2): если выбранная очередь переполнена, покупатель пробует вторую. В результате даже стратегия FIRST, которая всегда направляет покупателя в первую очередь, не приводит к его массовому уходу — при заполнении первой очереди он автоматически переходит во вторую. Разница между «умным» (SHORTEST) и «глупым» (FIRST) выбором нивелируется на этапе переключения.

2. **Стратегия RANDOM оказалась немного хуже** (17 обслужено, 33 ушло). При случайном выборе покупатели чаще попадают в уже заполненную очередь и, следовательно, чаще уходят, не сумев переключиться (обе очереди могут быть полны в моменты пиковой нагрузки).

3. **Загрузка кассиров равномерная во всех стратегиях** — благодаря переключению очередей оба кассира обслуживают сопоставимое количество покупателей (9/10 или 8/9). Без механизма переключения разница была бы заметной: при FIRST кассир 0 был бы перегружен, кассир 1 — простаивал.

Вывод: механизм переключения очередей (требование варианта 8) является ключевым фактором, обеспечивающим стабильную пропускную способность супермаркета независимо от выбранной стратегии. Если бы переключение не было реализовано, разница между стратегиями была бы существенной (как в последовательной версии ИДЗ-1, где FIRST давала 12% эффективности против 90% у SHORTEST).

Примечание о недетерминизме: многопоточная модель не является полностью детерминированной даже при фиксированном `random_seed` , поскольку порядок выполнения потоков зависит от планировщика ОС. При повторных запусках с одной и той же конфигурацией число обслуженных покупателей может отличаться на 1–2. В таблице приведены результаты одного из типичных прогонов.

6.3. Результаты тестирования

Автоматизированный тестовый набор включает 18 тестов:

- 13 позитивных тестов (различные конфигурации, граничные случаи, стресс-тесты);
- 5 негативных тестов (невалидные параметры, отсутствующий файл).

Все 18 тестов завершаются успешно. Закон сохранения покупателей соблюдается во всех сценариях.

---

7. Реализация альтернативных синхропримитивов

Для выполнения требования о реализации программы с использованием других синхропримитивов вместо семафоров, была применена условная компиляция.

В Makefile настроены две цели сборки:

- supermarket (использует семафоры POSIX sem_t).
- supermarket_condvar (использует условные переменные pthread_cond_t).

В исходном коде (types.h, simulation.c, threads.c, main.c) с помощью директив #ifdef USE_CONDVAR реализовано ветвление логики синхронизации. Это позволило создать два различных исполняемых файла на основе единой кодовой базы, что демонстрирует владение различными механизмами синхронизации потоков и соблюдение принципа DRY (Don't Repeat Yourself). Обе версии программы проходят все 18 автоматизированных тестов.

---

8. Выводы

В ходе выполнения задания разработана многопоточная программа моделирования работы супермаркета на языке C. Реализованы следующие ключевые механизмы:

1. Многопоточная архитектура: каждая сущность (покупатель, кассир, генератор, дисплей) реализована как отдельный поток POSIX.
2. Синхронизация: корректное взаимодействие потоков без гонок данных с использованием мьютексов и семафоров/условных переменных.
3. Потокобезопасный генератор случайных чисел: защита rand() через мьютекс.
4. Многопоточный табличный интерфейс: обновление в реальном времени через отдельный поток дисплея.
5. Ввод-вывод через системные вызовы: open, read, write, close.
6. Корректная обработка прерываний: через сигнал SIGINT с использованием async-signal-safe функций.
7. Автоматизированное тестирование: 18 тестов, все проходят, инварианты соблюдаются.

8. Корректная реализация условия варианта 8: при заполнении одной очереди покупатель переходит во вторую; уход — только если обе заполнены.
9. Ограничение рабочего дня через work_day_duration: генератор прекращает создавать новых покупателей при достижении порога.

Программа корректно завершается при любых входных данных, включая невалидные конфигурации и прерывание пользователем. Результаты моделирования показали, что благодаря реализованному механизму переключения очередей (требование варианта 8) стратегии SHORTEST и FIRST демонстрируют сопоставимую пропускную способность — покупатель, не сумевший встать в выбранную очередь, автоматически переходит во вторую. Стратегия RANDOM немного уступает (17 против 19 обслуженных), так как при случайном выборе покупатели чаще попадают в уже заполненные очереди. Переключение очередей также обеспечивает равномерную загрузку обоих кассиров независимо от стратегии.
